# HuAIBench: A Tiered Benchmark for Code-Failure Outcomes and Labeler Calibration

Anonymous Author(s)
*Submitted for review*

***Abstract*—Benchmarks for code generation usually measure whether programs pass tests; they rarely provide reusable evidence about how human and generated submissions fail. We introduce *HuAIBench*, a tiered benchmark for competitive-programming code failures: 14,182 sandbox-confirmed non-AC C++ submissions on 107 proxy-cutoff-aligned CodeContests problems, aligned across human submissions, a cutoff-aligned proxy LLM, a modern contrast LLM, and a public-example self-reflection loop. HuAIBench is tiered rather than a labeled corpus: it provides exact sandbox verdicts, the 32-leaf Wei et al. taxonomy as a common vocabulary, a 375-bug doubly annotated common-evidence gold set for labeler evaluation, LLM-judge baseline predictions released as system outputs rather than benchmark truth, a 120-item stronger-evidence audit with failing tests and reference solutions, and reflection trajectories with a same-budget resampling baseline. This structure turns the dataset into benchmark tasks with explicit evidence levels: sandbox outcomes, common-evidence triage labeling, stronger-evidence stress tests, and repair baselines. Exact verdicts support outcome, difficulty, and repair claims; human gold labels support taxonomy-labeler calibration; full-corpus LLM predictions support only exploratory coarse screens; and stronger-evidence packs stress-test diagnostic-label stability. The taxonomy task is triage under fixed evidence, not root-cause recovery. Baselines show the boundary: stronger evidence changes 55.0% of audited primary families, the judge over-applies design by 6–9 points, the only robust taxonomy signal is a coarse math/design bucket plus syntax/compile separation, and public-example self-reflection repairs only 4.0% of proxy bugs, matching resampling. We release the benchmark data, prompts, cached outputs, human labels, baseline predictions, audit packs, and analysis scripts.**

***Index Terms*—large language models, competitive programming, benchmark, bug taxonomy, LLM-as-a-judge, data contamination**

## I. Introduction

Code-generating large language models (LLMs) have moved from toy benchmarks to competitive programming, a domain demanding precise algorithmic reasoning, careful edge-case handling, and strict conformance to input/output (I/O) formats [1], [2]. Most work measures *whether* models solve such problems, via pass@$k$ on benchmarks like HumanEval, MBPP, and APPS [3]–[6]. Far less is known about *how* models fail, and even less about which failure claims are justified by the evidence available at scale. This is a benchmark gap: the field has many pass-rate benchmarks, but few artifacts that align human and generated failures on the same tasks, under the same sandbox, with calibrated taxonomy evidence.

The gap matters because taxonomies are already used as measurement instruments. If a taxonomy identifies stable failure regions, it can guide review, testing, and education for AI-assisted development. If labels instead reflect missing tests, over-broad categories, or an automated judge's granularity, then a precise-looking table can mislead. Calibrating such claims is hard for three reasons. First, competitive-programming benchmarks suffer from *data contamination*: problems and editorials appear in pre-training data, so apparent failures may reflect memorization rather than reasoning [2], [7]. Second, a useful calibration needs the *same* problems and a common vocabulary across human and generated failures, while still avoiding a false behavioral equivalence between curated human submissions and fixed-budget model samples. Third, labeling tens of thousands of buggy programs into a fine-grained taxonomy is beyond manual reach, pushing studies toward automated (LLM) labeling whose reliability is itself unestablished. A useful benchmark must therefore include not only programs and labels, but also the evidence needed to bound those labels' reliability.

We introduce *HuAIBench* to address this measurement gap. It adopts the recent bug taxonomy of Wei et al. [2]—six general-error and six algorithm-specific families, 32 leaves (Table I)—and bounds proxy-arm contamination risk by using the test split of CodeContests [1]: 107 public Codeforces problems first published after the September-2021 cutoff of `gpt-3.5-turbo` [8]. We use four sources as a stress test for the instrument: human submissions; a cutoff-aligned proxy model; a modern model whose training window covers the problems (a descriptive contrast, not a cutoff-controlled source); and a self-reflection loop on the cutoff proxy. Every submission is judged in a hardened sandbox; two annotators label a 375-bug gold set; an LLM judge produces full-corpus baseline predictions; and a 120-item audit adds failing tests and reference solutions. The current model arms are baseline instantiations. HuAIBench is not a contamination-proof leaderboard for future public models; it is a static measurement substrate for cached failures, labeler calibration, diagnostic-stability checks, and repair baselines.

We evaluate HuAIBench through four questions, ordered from benchmark validation to baseline findings: **RQ1** asks how reliable taxonomy labels and baseline predictions are under common evidence, automated labeling, and stronger failure evidence; **RQ2** asks which source-conditioned taxonomy screens remain after calibration; **RQ3** asks how capability, contamination risk, and difficulty shape label-free outcomes; and **RQ4** establishes a public-example self-reflection baseline.

HuAIBench's main output is a set of calibrated benchmark tasks, not an unconditional taxonomy or model leaderboard.

Researchers can evaluate common-evidence taxonomy labelers against the 375-bug human gold, stress-test diagnostic labels on the 120 stronger-evidence packs, compare repair methods against reflection/resampling trajectories, and study cached generator failures while reporting contamination status. Our baseline instantiation shows which uses are supported: exact verdicts are stable sandbox outcomes; human gold labels support triage-labeler calibration; full-corpus predictions are baseline screens; and root-cause claims require stronger evidence. The defensible taxonomy signal is coarse mathematics/design concentration plus a source-conditional syntax/compile separation. Label-free dynamics are stronger: the modern contrast is more capable but difficulty-sensitive, and public-example feedback repairs only 4% of proxy-model bugs, matching same-budget resampling.

Contributions: **(i)** *HuAIBench*, a released tiered measurement benchmark with 14,182 sandbox-confirmed non-AC programs and exact verdicts, 375 human-gold taxonomy labels, 14,182 full-corpus LLM baseline predictions, stronger-evidence audit packs, and repair trajectories; **(ii)** benchmark tasks and baselines for sandbox outcomes, common-evidence triage-labeler calibration, diagnostic-stability stress testing, and self-reflection; **(iii)** a validation protocol combining human gold, power analysis, problem-level bootstrap, per-family judge error, bias correction, and stronger-evidence relabeling; and **(iv)** baseline findings that define the current supported claim boundary.

## II. Background and Related Work

### A. LLMs for code and competitive programming

Transformer language models [9], [10] pre-trained on source code now span open and proprietary families [3], [8], [11]–[16], and a broad literature surveys their software-engineering uses [17]. Competitive programming is repeatedly singled out as the hardest target because it couples non-trivial algorithmic reasoning with strict resource and I/O-format constraints [1], [2]: AlphaCode [1] reached competition-level generation only through massive sampling and filtering. Most evaluation reports *whether* a model succeeds—pass@$k$ on HumanEval, MBPP, APPS, and successors [3]–[5], [18], or repository-level tasks like SWE-bench [19], drawing on code corpora and contest archives [20]–[22]—and rigorous re-testing shows such pass rates are often optimistic [6]. HuAIBench complements pass-rate benchmarks with aligned failure artifacts and asks what can be learned about *how* failures occur, and what evidence is needed before taxonomy labels become defensible measurements rather than diagnostic guesses.

### B. Data contamination

Because contest problems and editorial solutions are public, they routinely enter pre-training corpora and inflate benchmark scores; a growing line of work measures and mitigates this [23]–[26]. LiveCodeBench [7] adopts rolling, recent problems and reports strong models near zero on the hardest unseen tasks, and Wei et al. [2] run an explicit contamination audit. We take the same stance structurally: HuAIBench's CodeContests test split post-dates the `gpt-3.5-turbo` cutoff used for its proxy arm, and it adds a modern model as a deliberately contamination-uncontrolled contrast rather than a cutoff-controlled measurement.

### C. Bug taxonomies and empirical bug studies

Defect classification has a long lineage—Orthogonal Defect Classification [27], empirical bug characterizations [28]–[30], and curated fault benchmarks [31], [32] used to study repair [33]—almost all on *human* code; security studies of AI code (e.g., Copilot [34]) examine vulnerabilities rather than a general bug taxonomy. Closest to us, Wei et al. [2] recently proposed the taxonomy we adopt and hand-labeled 75 incorrect programs from a single model with strong inter-annotator agreement ($\kappa = 0.86$), using *no* automated labeler and providing *no* human baseline. HuAIBench reuses their taxonomy verbatim but differs in kind as much as scale: it releases **14,182** sandbox-confirmed non-AC submissions from *four* sources—including *human* submissions to the *same* problems—under explicit cutoff alignment and contamination caveats, with full-corpus LLM predictions treated as a baseline system output whose reliability is validated against a 375-bug doubly-annotated human gold. The human baseline lets the benchmark calibrate source-conditioned screens on identical problems, and the scale forces measurement questions—statistical power, within-problem clustering, and labeler granularity—that a 75-program, single-model, AI-only study never has to confront.

### D. Automated repair and self-correction

Automated program repair has matured from search- and learning-based methods [32], [33] surveyed broadly [35], [36] to LLM-based repair [37]. For models repairing their *own* output, Self-Refine [38] critiques and revises with self-feedback, self-debugging [39] and Reflexion [40] add execution or verbal feedback, and Olausson et al. [41] question whether self-repair is a "silver bullet." HuAIBench includes a reflection loop with the public example-test results a contestant actually sees, and ask whether repair changes the *type* of remaining bugs rather than only the pass rate.

### E. LLM-as-a-judge and agreement methodology

Using an LLM to score or label outputs is now widespread [42], [43], but it carries known biases [44] and its reliability for fine-grained, domain-specific taxonomies is rarely audited. HuAIBench treats the judge as a measurement instrument: we size the human gold by Cochran's proportion formula [45], [46], report chance-corrected agreement and its interpretation [47]–[51], and follow empirical-software-engineering guidance [52], [53].

### F. The taxonomy we adopt

We do not propose a taxonomy; we reuse Wei et al.'s [2] verbatim (Table I) so our labels are comparable to theirs. It has two top levels. *General Errors* (GE) describe *how* a program is wrong regardless of the algorithm: design/wrong-algorithm (GE1), boundary and off-by-one (GE2), condition

TABLE I
THE TAXONOMY OF WEI ET AL. [2]: 6 GENERAL-ERROR (GE) AND 6 ALGORITHM-SPECIFIC (AE) FAMILIES, 32 LEAVES.

| Code | Family (#leaves) |
|---|---|
| GE1 | Design: wrong algorithm, misread requirement (4) |
| GE2 | Boundary: edge cases, off-by-one/indexing (2) |
| GE3 | Condition: predicates, operator/precedence (2) |
| GE4 | Data type: overflow, implicit conversion (2) |
| GE5 | Syntax: compile errors, language misuse (2) |
| GE6 | Input/Output: parsing, output formatting (2) |
| AE1–AE6 | Math, greedy, graph, recursion/D&C, DP, search (18) |

TABLE II
SANDBOX VERDICTS. A *bug* IS ANY NON-AC SUBMISSION; THE VERDICT IS THE MOST SEVERE OUTCOME OVER ALL TESTS.

| Verdict | Meaning |
|---|---|
| AC | Accepted: correct output within limits on every test |
| WA | Wrong Answer: output mismatches the expected output |
| TLE | Time-Limit Exceeded: exceeds the problem's CPU budget |
| RE | Runtime Error: crash, non-zero exit, or sandbox fault |
| CE | Compile Error: source fails to build with `g++` |

and operator mistakes (GE3), data-type issues such as integer overflow (GE4), syntax/compile errors (GE5), and input/output formatting (GE6). *Algorithm-specific Errors* (AE) describe *which* algorithmic technique was misapplied: mathematics (AE1), greedy (AE2), graph (AE3), recursion/divide-and-conquer (AE4), dynamic programming (AE5), and search (AE6). Each of the 12 families is split into leaves—e.g. GE1 separates *Incorrect Algorithm*, *Misunderstanding Requirements*, and *Inefficient design*—for 32 leaves total. Labeling is multi-label, since one program can exhibit several independent errors. The GE/AE split is the axis our analysis returns to: GE5/GE4 are coding errors a tool can catch, whereas GE1 and the AE families are reasoning errors that require understanding the problem.

## III. STUDY DESIGN

Figure 1 summarizes HuAIBench as four panels. The input panel filters CodeContests problems and collects human, proxy-LLM, modern-LLM, and reflection submissions. The sandbox panel assigns shared verdicts. The taxonomy panel combines the fixed Wei et al. taxonomy with human gold labels, LLM-judge baseline predictions, and a stronger-evidence audit. The analysis panel calibrates which verdict, family, and root-cause claims the evidence can support.

### A. HuAIBench construction: tasks, sandbox, and submissions

We use the CodeContests [1] *test* split: Codeforces problems first published after 2021-09-21, conservatively after `gpt-3.5-turbo`'s September-2021 cutoff [8]. We treat this cutoff as necessary but not sufficient—the exact training window of the `-0125` snapshot is not publicly documented per-release—so we add a behavioral sanity check: the cutoff proxy's acceptance is low and *flat* across difficulty ($\leq 8\%$; Sec. IV-C), which is consistent with solving from the statement rather than recalling seen tasks. We also run a lightweight code-similarity audit against the stored human accepted solutions for each problem: none of the proxy model's 64 accepted outputs is whitespace-normalized identical to a stored human accepted solution, and the maximum token 5-gram Jaccard similarity is 0.47 (median 0.20). This does not prove absence of training exposure, but it rules out verbatim or near-verbatim replay of the accepted solutions available in our corpus. Throughout, we therefore call this arm the *cutoff proxy*, not a certified contamination-free model. Every cutoff-aligned taxonomy claim rests on this proxy; we make none on the modern model. We start from 165 Codeforces test-split problems and keep a problem only if it is judgeable ($\geq 1$ correct human C++ solution confirmed AC and $\geq 10$ incorrect C++ solutions confirmed non-AC). The filter discards 44 problems with no private tests, 7 whose stored correct C++ solutions did not pass our sandbox comparator, and 7 with too few confirmed human bugs, leaving **107 problems** (ratings 800–3500). This gate is also our guard against special-judge or multiple-output mismatches: we do not run Codeforces custom checkers, so problems for which known-correct code cannot be accepted under our token/float comparator are excluded. During filtering, stored incorrect solutions that unexpectedly pass our tests are quarantined as sandbox-inconclusive pass-throughs, not counted as bugs (3,690 of 12,356 judged incorrect C++ submissions). For retained problems, per-problem counts are released as oracle-risk metadata; they flag weak-test or multiple-output risk and bound the outcome task to "verdict under the released sandbox tests," not semantic correctness in the abstract. The set is deliberately hard-skewed—25 problems rated $\leq$1199, 12 at 1200–1599, 13 at 1600–1999, 22 at 2000–2399, and 35 rated 2400+—so RQ3's difficulty analysis spans the full competitive range rather than only easy tasks. Each C++ submission is compiled with GNU `g++` (`gnu++17`) and run under an OS sandbox (no network, bounded CPU/memory/stack/file size, process-group timeouts) on public and private tests; output comparison is whitespace-tokenized, case-insensitive, $10^{-4}$ float tolerance. Each run yields one of five verdicts (Table II); when a submission fails differently on different tests we report the most severe under the precedence $\mathsf{CE} > \mathsf{RE} > \mathsf{TLE} > \mathsf{WA} > \mathsf{AC}$ (a program that does not compile cannot be timed, one that crashes cannot be judged for output, and so on), cached by `(problem, source, tests)`. A bug is any submission whose verdict is not AC.

**Human** solutions come labeled in CodeContests; we re-judge all and keep only label-consistent ones, yielding **8,648 human bugs** (and 9,925 confirmed-correct). **AI** solutions are generated zero-shot from a PaR-style prompt [2] (task framing plus the verbatim statement, which carries the I/O format and examples) at temperature 1.0 with a *fixed* budget of 20 samples per problem (no early stop): `gpt-3.5-turbo-0125` (proxy) gives 64 AC and **2,076 bugs**; `gpt-5.4-nano` (modern) gives 675 AC and **1,465 bugs**. We stress that

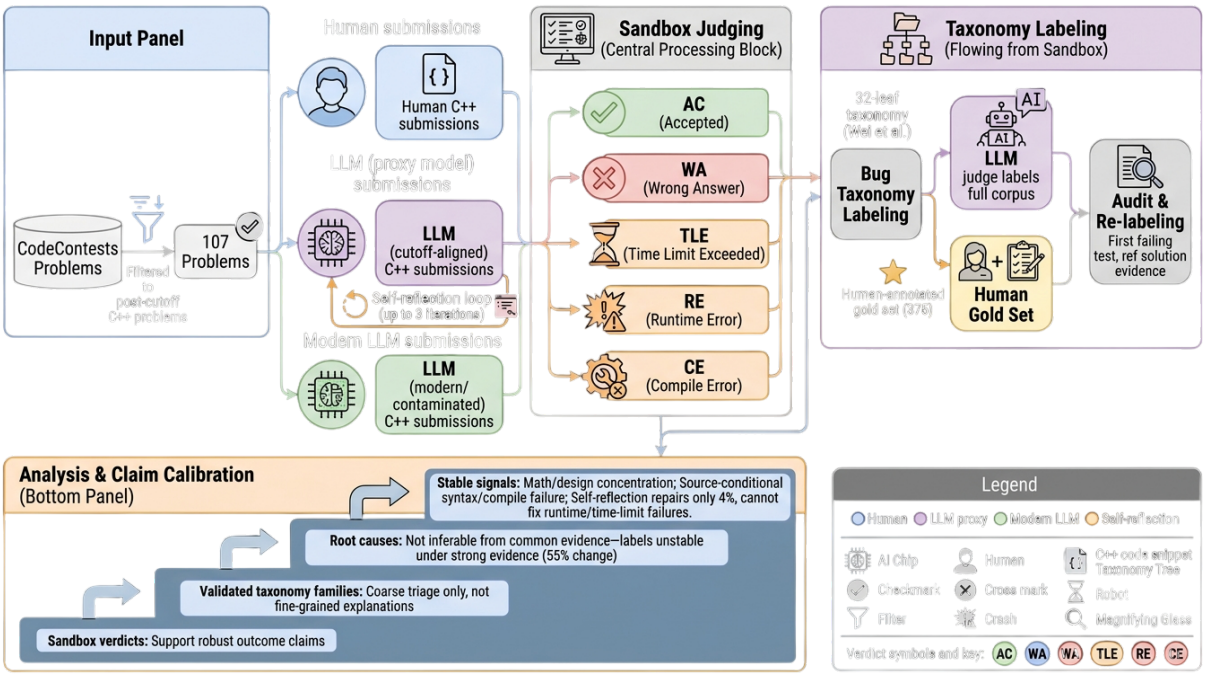


Fig. 1. Overview of HuAIBench: filtered human/generated submissions are rejudged in one sandbox, labeled with the Wei et al. taxonomy through human gold and LLM-judge baseline predictions, audited with stronger evidence, and interpreted through claim calibration.

TABLE III
BUG CORPUS OVER 107 PROBLEMS. THE SUCCESS METRIC IS NOT COMPARABLE ACROSS HUMAN AND AI: HUMAN IS A CURATED CORRECT:INCORRECT RATIO; AI IS PER-SAMPLE PASS RATE.

| Source | Confirmed bugs | Success metric |
|---|---|---|
| Human | 8,648 | 53.4% |
| `gpt-3.5` proxy | 2,076 | 3.0% |
| `gpt-5.4-nano` modern | 1,465 | 31.5% |
| `gpt-3.5` reflect | 1,993 | — |
| Total | 14,182 | |

`gpt-5.4-nano` is *not* a cutoff-controlled source: its training window covers these problems, so it cannot isolate reasoning from memorization. It is included for a different purpose—to stress-test labeler and sample effects on a present-day arm—and `gpt-3.5-turbo-0125` alone carries the cutoff-aligned claims. Every cutoff-aligned taxonomy result in this paper rests on this proxy model; the modern model is descriptive context, read always against that caveat. **Self-reflection** applies up to three Self-Refine rounds [38] per proxy-model bug, continuing the same conversation with the public example-test results, stopping at AC; **1,993** bugs survive. The total is **14,182 sandbox-confirmed bugs** (Table III).

*a) Estimand and non-goals:* The human and AI corpora are not interchangeable behavioral samples. Human submissions are curated CodeContests artifacts with separate correct and incorrect pools; AI submissions are fixed-budget samples from a prompt. We therefore do not use the human correct:incorrect ratio and AI per-sample pass rate to compare programmer ability. The taxonomy estimand is narrower and conditional: *among sandbox-confirmed non-AC C++ submissions on the same problems*, how are bug mechanisms distributed? Acceptance rates are used only for within-source trends (e.g., by difficulty) and for the proxy-vs-modern contamination/capability contrast. This estimand does not claim that a curated human incorrect submission and a zero-shot LLM sample arise from the same behavioral process.

## B. Prompts and protocols

We summarize the three prompt-driven protocols; full prompts are released. *Generation* follows the PaR template [2]: a system message frames the task ("write a correct, efficient C++ program; return only a code block"), and the user message is the verbatim problem statement, which on Codeforces already contains the I/O specification and worked examples; no unit tests or scaffolding are added, so the model must reason from the statement alone. The fixed-budget choice—exactly 20 samples per problem, no early stop—equalizes effort so that a model's bug yield reflects its true failure rate rather than a collection cap; the proxy model yields ≈19 bugs per problem, the modern model ≈14. *Self-reflection* continues the same conversation: we append the model's own buggy program as the assistant turn, then a user turn reporting its results on the public example tests (the input, the expected output, and the program's actual output for each), and ask for a corrected program, for up to three rounds, stopping at the first AC. The model thus sees exactly the feedback a contestant reads from the sample tests, and nothing more. *Judging* gives `gpt-5.4` the problem statement, the buggy

code, and the verdict, and asks for the applicable taxonomy leaves plus a one-line rationale in strict JSON, sampled three times with majority aggregation. All model calls use the OpenAI Chat Completions API through the model strings named above; generation/reflection use temperature 1.0, the judge uses temperature 0.4, and no `top_p`, frequency, or presence penalties are set. Responses are cached by model, temperature, prompt, and nonce; because proprietary model strings are not immutable snapshots, the released raw generations, judge outputs, and cache keys define the audit record. The API calls that produced the released generation/reflection corpus were completed on 2026-06-29, and the judge predictions on 2026-06-30. Exact re-generation is not promised; reproducibility means replaying the analyses from released prompts, outputs, labels, verdicts, and scripts. The judge is provenance-blind and receives neither the failing hidden test nor a reference solution, matching the human annotators' information exactly so that disagreement reflects judgment, not evidence. This is a deliberate choice for an apples-to-apples agreement study, but we acknowledge its cost: inferring *which* algorithmic family is at fault for a wrong answer from code alone is genuinely hard, and that irreducible diagnostic noise is itself a plausible contributor to the fine-grained instability we document in RQ1. We therefore keep the common-evidence protocol for the full corpus, and add a separate first-failing-test audit to stress-test whether diagnostic labels remain stable under stronger evidence.

### C. Labeling and analysis

Labeling is multi-label into the 32 leaves [2]. The released annotation packets include the fixed codebook with every family/leaf definition; no model- or source-derived label hints appear in the packets. We size the gold by Cochran's formula with finite-population correction [45], [46]: 95% confidence, $\pm 5\%$ margin on 14,182 items gives $n \approx 375$. We allocate it arm-balanced (125 each: human, proxy, modern), stratify by verdict so rare verdicts appear, and cap four items per problem. Annotators used the fixed taxonomy with the same decision rules in both gold and audit packs: prefer the most specific leaf that explains the demonstrated failure, multi-label only for independent causes, and do not label downstream symptoms. Primary-family tables use the first leaf after this rule. **Two annotators** labeled all 375 items provenance-blind (problem statement and buggy code only); a third adjudicated; we report their inter-annotator agreement. The **LLM judge** (`gpt-5.4`) produces baseline predictions for all 14,182 bugs from the same evidence (statement, code, verdict) with self-consistency over three samples. For verdict-based questions (acceptance, difficulty, repair) we use the exact full corpus; for taxonomy-distribution questions we use the human gold and validate the judge against it (RQ1). We report a descriptive $\chi^2$/Cramér's $V$ on family *occurrences*: each multi-label bug contributes one count to every family present. Nine families appear at least once in the zero-shot corpus (GE1–GE6, AE1, AE2, AE5), giving $\mathrm{df} = 8$ for pairwise source comparisons; AE3/AE4/AE6 are absent and the GE3 singleton is kept in the test but omitted from the compact table. We do not use this anti-conservative test for inference. Instead, we use a problem-level bootstrap because bugs are multi-label and clustered within problems. For each replicate we sample 107 problem IDs with replacement; within each sampled problem and source, we compute the share of that source's non-AC bugs whose labels contain each family, skipping problem-arm cells with no bugs; then we average the model–human difference over sampled problems. We report percentile 95% intervals from 2,000 replicates. This estimates an equal-problem conditional bug-profile difference, rather than letting high-volume problems dominate the estimate.

Three choices determine what the calibration can support. The gold is *arm-balanced* (125 bugs per source) rather than proportional, trading population precision for equal source power. Verdict stratification with rare-verdict floors keeps compile, runtime, and time-limit failures estimable. All packages are *provenance-blind*: statement, code, and verdict only, with author identity stripped and within-problem order shuffled. Statistically, we treat the judge as an instrument whose error we bound against the human ceiling (RQ1), not as ground truth.

Finally, after the main analysis, two annotators independently re-labeled a stratified 120-item wrong-answer audit set (40 per zero-shot source) with stronger evidence: the same statement and code plus the first failing test's input, expected/actual output, and one accepted reference implementation. If a program failed a public sample first, that public sample is the first failing test; otherwise the evidence is private-test based (95 public, 25 private overall: 18/40 public for human, 40/40 for proxy, 37/40 for modern). The $n$=120 audit summarizes the overall family-change rate with about $\pm 9$ percentage points of binomial precision; Wilson 95% intervals are reported with the audit counts in RQ1. It is not designed for per-family prevalence. The annotators adjudicated all disagreements, with a third tie-breaker for three items. This audit does not relabel the full corpus and is not an isolated evidence-effect experiment; it stress-tests how unstable common-evidence diagnostic labels can be when the same taxonomy is applied with stronger evidence and a root-cause target.

## IV. Results

### A. RQ1: How reliable are HuAIBench taxonomy labels?

The first result is about the instrument, not the model. The two human annotators on the 375-bug common-evidence gold agree strongly—Cohen's $\kappa = 0.80$ at leaf level and $0.83$ at family level—matching the high agreement reported for this taxonomy by Wei et al. [2]. The `gpt-5.4` judge reaches substantial agreement with both annotators (family $\kappa = 0.77/0.81$, micro-$F_1 \approx 0.77$; Table V), so it is usable for coarse full-corpus screening. It is not neutral enough for fine distributional claims. On the same 375 gold items the judge assigns the broad design family to 26/34/41% of human/proxy/modern bugs, while the human annotators assign 20/28/32%: a 6–9 point inflation that is arm-dependent and

TABLE IV
JUDGE MEASUREMENT-ERROR SENSITIVITY. P/R USE THE 315 GOLD ITEMS WITH ANNOTATOR-AGREED PRIMARY FAMILIES. DELTAS ARE PERCENTAGE POINTS: RAW→BIAS-CORRECTED, WITH 95% BOOTSTRAP CIS FOR CORRECTED DELTAS FROM THE 375-ITEM GOLD.

| Family | P/R | Proxy–H | Modern–H |
|---|---|---|---|
| GE1 Design | 74.5/96.3 | -6.7→-5.1 [-13.1,+2.5] | +0.9→-1.9 [-9.9,+6.1] |
| AE1 Math | 95.7/80.0 | +0.5→+0.9 [-5.9,+7.7] | -2.0→+6.8 [-0.8,+14.8] |
| GE2 Boundary | 76.9/95.2 | +1.7→+1.7 [-2.7,+6.1] | -1.9→-4.3 [-7.9,+0.5] |
| GE6 I/O | 100.0/66.7 | -1.2→-2.8 [-5.6,-0.8] | -1.2→-2.0 [-4.8,+0.8] |
| AE1+GE1 | – | -6.2→-4.2 [-10.2,+1.8] | -1.1→+4.9 [-1.5,+12.1] |

TABLE V
LABELING AGREEMENT ON THE 375-BUG COMMON-EVIDENCE GOLD (COHEN'S $\kappa$, MICRO-$F_1$).

| Pair | $\kappa$ (leaf) | $\kappa$ (family) | micro-$F_1$ |
|---|---|---|---|
| human1 vs. human2 (ceiling) | 0.80 | 0.83 | 0.80 |
| judge vs. human1 | 0.75 | 0.77 | 0.76 |
| judge vs. human2 | 0.77 | 0.81 | 0.77 |

largest on the modern arm, which shares the judge's model family.

Table IV makes this error structure explicit rather than relying on aggregate $\kappa$: GE1 has high recall but lower precision, AE1 has the opposite shape, and additive arm-specific bias correction leaves the focused family shifts small or direction-unstable. The corrected-delta intervals often include zero; even the combined AE1+GE1 proxy–human contrast is $-4.2$ points with a 95% bootstrap CI of $[-10.2, +1.8]$. The combined AE1+GE1 region remains large under correction (human 68.8%, proxy 64.6%, modern 73.7%), supporting coarse concentration but not an ordered source comparison.

The stronger test is evidence stability. In the 120-item audit, annotators relabeled wrong-answer submissions with the first failing test, actual/expected output, and an accepted reference implementation. They again reached substantial agreement (primary-family $\kappa = 0.69$; binary multi-label family $\kappa = 0.71$), then adjudicated all 36 disagreements. Against the original common-evidence labels, the adjudicated stronger-evidence labels changed 70/120 leaves (58.3%) and 66/120 families (55.0%; Table VI). The rate is not confined to one source: family labels changed for 65.0% of human submissions and 50.0% of both AI arms. We use this as a diagnostic-stability warning, not as a causal estimate of evidence alone.

Table VIII is the construct boundary for using HuAIBench. The benchmark has direct ground truth for released-test sandbox outcomes and repair trajectories, human ground truth for *common-evidence triage labeling* (given statement, code, and verdict, predict the label a provenance-blind human assigns), full-corpus baseline predictions for coarse family profiles, and a stronger-evidence stress set for diagnostic-label robustness. The judge predictions are released as a baseline system output, not as benchmark ground truth. A full-corpus taxonomy difference is not promoted beyond a screen unless it survives problem-level clustering and the arm-specific bias correction in Table IV; root-cause claims are never made from common evidence alone. The 55.0% should not be read as a pure evidence effect: the comparison changes evidence, task target, and adjudicated label source at the same time. The stronger-evidence task is also harder, with lower annotator agreement than the common-evidence gold (family $\kappa = 0.69$ vs. 0.83), so the number combines evidence gain, taxonomy-boundary ambiguity, and root-cause-labeling instability. Nor do we claim the stronger-evidence label is uniquely true: one failing test and one reference solution can still leave multiple plausible fixes. The audit's role is to stress-test label stability under better evidence, not to certify final root causes. The transition summary (Table VII) shows where this instability

TABLE VI
FIRST-FAILING-TEST/REFERENCE-SOLUTION RELABELING AUDIT ON 120 ZERO-SHOT WA SUBMISSIONS (40 PER SOURCE). THE TOP BLOCK IS INTER-ANNOTATOR AGREEMENT WITHIN THE STRONGER-EVIDENCE TASK; "CHANGED" ROWS COMPARE THE ADJUDICATED STRONGER-EVIDENCE LABEL WITH THE ORIGINAL COMMON-EVIDENCE LABEL. WILSON INTERVALS ARE FOR DESCRIPTIVE BINOMIAL PROPORTIONS, NOT INDEPENDENT CAUSAL EFFECTS.

| Metric | Count | Value | 95% CI |
|---|---|---|---|
| Leaf exact agreement | 84/120 | 70.0% | – |
| Family exact agreement | 85/120 | 70.8% | – |
| Primary-family $\kappa$ | – | 0.69 | – |
| Binary family $\kappa$ | – | 0.71 | – |
| Adjudicated disagreements | 36/120 | 30.0% | – |
| Leaf changed | 70/120 | 58.3% | [49.4,66.8] |
| Family changed | 66/120 | 55.0% | [46.1,63.6] |
| Math/design before | 44/120 | 36.7% | [28.6,45.6] |
| Math/design after | 58/120 | 48.3% | [39.6,57.2] |
| Bucket stable | 84/120 | 70.0% | [61.3,77.5] |
| Into math/design | 25/120 | 20.8% | [14.5,28.9] |
| Out of math/design | 11/120 | 9.2% | [5.2,15.7] |

TABLE VII
PRIMARY-FAMILY TRANSITIONS IN THE 120-ITEM STRONGER-EVIDENCE AUDIT. ROWS SHOW ORIGINAL COMMON-EVIDENCE FAMILY; DESTINATIONS LIST THE LARGEST STRONGER-EVIDENCE FAMILIES AFTER ADJUDICATION. SINGLETON ORIGINAL GE3 IS OMITTED.

| Orig. | $n$ | Same | Main destinations |
|---|---|---|---|
| GE1 | 21 | 11 | AE1 6; AE6/GE3/AE5/GE2 1 each |
| GE2 | 20 | 2 | GE1 8; GE3 5; AE1 3 |
| GE4 | 10 | 1 | AE1 5; GE2 2 |
| GE6 | 8 | 2 | GE1 4; AE1/GE2 1 each |
| AE1 | 22 | 15 | AE5 4; GE2 2 |
| AE2 | 25 | 19 | GE3 3; GE1 2 |
| AE5 | 13 | 7 | GE1 3; GE2 2 |

TABLE VIII
HUAIBENCH BENCHMARK TASKS AND CLAIM GATES. EACH TASK HAS RELEASED EVIDENCE, A BASELINE METRIC IN THIS PAPER, AND A SUPPORTED CLAIM LEVEL.

| Task | Released evidence | Baseline metric | Claim gate |
|---|---|---|---|
| Sandbox outcome | verdicts, tests, cache | pass/verdict by problem | sandbox direct |
| Triage labeler | 375 human gold labels | $\kappa$, $F_1$, bias | common evidence |
| Corpus screen | 14,182 judge predictions | bootstrap + bias | coarse only |
| Evidence stress | 120 stronger packs | stability, transitions | diagnostic only |
| Repair | reflection trajectories | fixed vs. resampling | repair baseline |

concentrates: boundary, data-type, and I/O diagnoses are often reinterpreted as design, condition, or mathematics, while AE1 and AE2 are more stable. The audit also checks whether our coarsest claim survives stronger evidence. Its sample was deliberately stratified rather than population-representative, so it cannot estimate the full-corpus prevalence of math/design bugs; it only checks whether stronger evidence obviously erodes that bucket. Membership in {mathematics, design} is stable for 84/120 items (70.0%, [61.3,77.5]), and the crossings are net inward: 25 items move into the bucket and 11 move out. Family-set changes are not a single-arm artifact (65.0% human, 50.0% proxy, 50.0% modern) or a hard-problem artifact (65.2/50.7/57.7% for easy/medium/hard), but they are descriptively more common when the first failing evidence is private rather than public (72.0% [52.4,85.7] vs. 50.5% [40.6,60.4]). We therefore treat fine shifts as exploratory screens and reserve strong claims for bounded coarse patterns and label-free verdict dynamics.

### B. RQ2: What calibrated source screens remain?

RQ2 is a benchmark sanity check: it asks what a user would see when applying the released baseline predictor under the claim gates, not whether humans and LLMs have intrinsic failure distributions. We treat family distributions as screens unless a signal is larger than known labeler bias and consistent with the audit's coarse stability check. On the human gold (Table XI) all three sources are dominated by the same two families—mathematics (AE1, ≈33%) and design (GE1, 20–32%)—and their high-level general-vs-algorithmic split is similar without being diagnostic. A *weak* directional tendency is visible: design (GE1) runs 20% (human) / 28% (proxy) / 32% (modern), while implementation boundaries (GE2: 11/7/5%) and I/O formatting (GE6: 6% vs. $<1\%$) run the other way. But the tendency is, on the gold, *underpowered*. At 125 bugs per arm the three-way family $\chi^2$ can detect only $V \gtrsim 0.26$ at 80% power (a minimum-detectable-effect we compute by inverting the noncentral $\chi^2$), and the observed $V \approx 0.23$ ($p \approx 0.07$–$0.10$ per annotator) sits just below that floor—so the gold can neither confirm the tendency nor rule it out, and we do not read its non-significance as evidence of no effect.

*a) Full-corpus shifts are small, not self-interpreting:* The full corpus helps with precision but not automatically with validity. A descriptive $\chi^2$ on the source×family occurrence table over the nine observed families (Sec. III; $\mathrm{df} = 8$) gives high significance at a negligible effect size—Cramér's $V = 0.11$ (human–proxy, $p = 3 \times 10^{-25}$), 0.08 (human–modern)—and the problem-level bootstrap does detect several small shifts (Table IX). The proxy model has fewer design labels than humans ($-5.6$ points) and more mathematics/syntax labels (+4.0/+2.6); the modern model shows smaller shifts, with syntax and mathematics up by about 2–3 points and boundary/I/O down by about 1–2 points. These are not large taxonomy-defining separations: all major shifts are within the 6–9 point design inflation that RQ1 finds in the judge itself, and most are much smaller. We therefore use the judge-labeled full corpus as a *screen* for fine shifts, not as a causal claim that curated

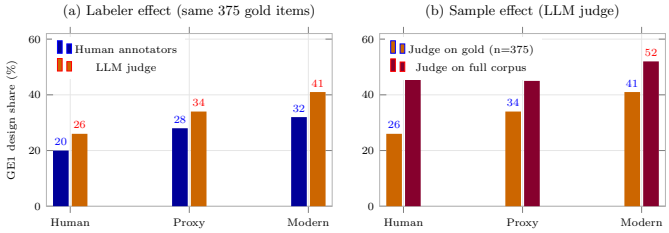


Fig. 2. Deconfounding the apparent design (GE1) reversal. **(a)** Holding the *sample* fixed (the same 375 gold items), the LLM judge inflates the design family (∼+6–9 points) but *preserves* the human<proxy<modern ordering—its granularity bias is a level shift, not a reversal. **(b)** Holding the *labeler* fixed (judge), moving from the stratified gold to the full corpus changes the ordering (proxy drops to last): the "reversal" is a sample effect. Neither route yields a robustly ordered difference.

human submissions and fixed-budget model samples have different diagnostic distributions. The bias-corrected check in Table IV preserves only the bounded result: AE1+GE1 remains the dominant region for every arm, but source order is not stable. The low-level story is more qualified: data-type and I/O labels are rare everywhere, while syntax/compile failures remain a source-conditional separation.

*b) The apparent reversal is a sample effect, not a labeler effect:* A natural worry is that the comparison reverses under the judge: design runs 20/28/32 (human/proxy/modern) under the human annotators but 51/45/52 under the judge on the full corpus, the proxy model dropping to last. Holding the *sample* fixed dispels this (Fig. 2). On the same 375 gold items the judge gives 26/34/41—it *inflates* design by ∼6–9 points (a real granularity bias, quantified in RQ1) but *preserves* the human<proxy<modern ordering. The reversal appears only when the judge moves from the stratified gold to the natural corpus ($26/34/41 \rightarrow 51/45/52$), so it is driven by the *sample* (verdict-stratified, per-problem-capped gold vs. the full corpus), not the labeler. Neither sample nor labeler yields a stable ordered design story, which is why we treat the fine shifts as measurement-limited rather than explanatory.

The one structural regularity worth stating is what is *shared*: across both the gold and the full corpus, and across all three sources, the mass concentrates in the two "thinking" families (mathematics AE1, design GE1). This should not be mistaken for equality in low-level behavior. Syntax labels are higher for AI in the full corpus (GE5, +2.6/+2.9 points), and compile failures are more than twice as common for the models as for humans (Table XIII). Thus the careful statement is: low-level errors are minority modes for every source, but syntax/compile remains one of the clearest residual separations.

Nor does the coarse pattern eliminate the proxy-model floor confound. The cutoff proxy accepts only 3.0% of attempts, so many failures may be far from a near miss. We therefore report robustness checks as scope checks, not proof of equivalence. Table X shows that the AE1+GE1 concentration survives the WA-only slice and the stricter AC-problem slices, where a model has at least one accepted sample on the same problem. But the proxy AC-problem slice contains only 208 proxy

TABLE IX

FULL-CORPUS BASELINE JUDGE PREDICTIONS. PERCENT COLUMNS ARE CORPUS SHARES OF BUGS WITH A FAMILY PRESENT. Δ COLUMNS ARE EQUAL-PROBLEM MODEL–HUMAN DIFFERENCES WITH PROBLEM-BOOTSTRAP 95% CIS (2,000 REPLICATES). JUDGE-LABEL UNCERTAINTY IS HANDLED SEPARATELY BY THE BIAS SENSITIVITY IN TABLE IV; UNDER TABLE VIII, THESE ROWS REMAIN SCREENS.

| Family | Human | Proxy | Modern | Δ Proxy–H | Δ Modern–H |
|---|---|---|---|---|---|
| GE1 Design | 51.4 | 44.8 | 52.3 | $-5.6[-7.9, -3.4]$ | $-0.7[-3.9, +2.3]$ |
| GE2 Boundary | 8.2 | 9.9 | 6.3 | $-0.6[-2.0, +1.0]$ | $-2.3[-4.0, -0.4]$ |
| GE4 Data type | 1.0 | 0.1 | 0.1 | $-0.7[-1.5, -0.1]$ | $-0.7[-1.5, -0.1]$ |
| GE5 Syntax | 2.6 | 6.1 | 5.1 | $+2.6[+0.8, +4.6]$ | $+2.9[+0.7, +5.4]$ |
| GE6 I/O | 1.2 | 0.0 | 0.1 | $-1.5[-2.0, -1.0]$ | $-1.3[-1.9, -0.7]$ |
| AE1 Mathematics | 19.4 | 19.9 | 17.4 | $+4.0[+2.3, +5.8]$ | $+2.4[+0.9, +4.0]$ |
| AE2 Greedy | 7.7 | 9.2 | 10.1 | $+0.4[-0.4, +1.3]$ | $+0.8[-0.0, +1.5]$ |
| AE5 DP | 8.5 | 10.1 | 8.7 | $+1.3[+0.4, +2.4]$ | $-0.9[-2.9, +0.7]$ |
| Cramér's $V$: H–C 0.11, H–M 0.08, C–M 0.09 | | | | | |

TABLE X

SCOPE CHECKS FOR THE COARSE AE1+GE1 CLAIM. VALUES ARE % OF BUGS IN MATHEMATICS OR DESIGN; PARENTHESIZED COUNTS ARE BUGS IN THE SLICE. AC-PROBLEM SLICES KEEP ONLY WA BUGS FROM PROBLEMS WHERE THAT MODEL HAS AT LEAST ONE ACCEPTED SAMPLE.

| Slice | Human | Proxy | Modern |
|---|---|---|---|
| All non-AC | 70.8 (8648) | 64.6 (2076) | 69.7 (1465) |
| WA only | 73.8 (8181) | 70.4 (1868) | 73.6 (1370) |
| Proxy AC-problems | 85.4 (874) | 82.7 (208) | – |
| Modern AC-problems | 75.6 (5083) | – | 72.0 (779) |

TABLE XI

BUG-FAMILY DISTRIBUTION ON THE HUMAN GOLD (% OF LABELS, MEAN OF TWO ANNOTATORS); BOLD MARKS THE LARGER HUMAN/AI SIDE. THIS IS A HUMAN-LABELED GOLD-SAMPLE TENDENCY, NOT A DEFINITIVE POPULATION ESTIMATE.

| | Family | Human | Proxy | Modern |
|---|---|---|---|---|
| GE1 | Design / wrong algorithm | 20 | 28 | **32** |
| GE2 | Boundary / edge cases | **11** | 7 | 5 |
| GE5 | Syntax | 9 | 8 | 9 |
| GE6 | Input/Output | **6** | 1 | 1 |
| AE1 | Mathematics | 32 | 35 | 33 |
| AE2 | Greedy | 8 | **13** | 9 |
| AE5 | Dynamic programming | 8 | 6 | 8 |

bugs on easier problems, so it cannot rescue a fine source-comparison claim. It only supports the bounded statement that the defensible slices keep most mass in math/design rather than low-level coding families.

### C. RQ3: Capability, contamination, and difficulty outcomes

Table III reports three success metrics, but they answer different questions: the human number is a curated correct:incorrect ratio, whereas the AI numbers are fixed-budget per-sample pass rates. We therefore do not use the table as a cross-source ability comparison. The defensible signal is the *within-source* trend with difficulty (Table XII, Fig. 3). The proxy model is at the floor everywhere ($\leq$ 8%). The modern model is much stronger but not simply solved by lookup: it falls from 55.0% on the easiest band to 12.3% on the hardest, with a non-monotone middle band (21.2% then 35.0%) that points to problem-set heterogeneity rather than a smooth ability curve. Its high acceptance on some hard problems is consistent with partial contamination, but the

TABLE XII

PER-SUBMISSION ACCEPTANCE (%) BY DIFFICULTY BAND.

| Difficulty | Human | Proxy | Modern |
|---|---|---|---|
| ≤1199 | 59.3 | 7.6 | 55.0 |
| 1200–1599 | 49.5 | 0.8 | 43.8 |
| 1600–1999 | 42.9 | 0.4 | 21.2 |
| 2000–2399 | 55.5 | 3.6 | 35.0 |
| 2400+ | 53.3 | 1.0 | 12.3 |

TABLE XIII

VERDICT DISTRIBUTION PER SOURCE (% OF CONFIRMED BUGS, EXACT).

| Source | WA | TLE | RE | CE |
|---|---|---|---|---|
| Human | 94.6 | 0.9 | 1.8 | 2.6 |
| Proxy | 90.0 | 1.3 | 2.6 | 6.1 |
| Modern | 93.5 | 0.9 | 0.5 | 5.1 |

overall decline argues against pure memorization. The human rate is flat (42–59%), a survivorship artifact: harder problems attract stronger solvers.

The contrast between the two models is the contamination design. The proxy looks like a 2023-class model attempting unseen tasks from the statement; the modern arm may mix real capability gains and partial memorization of easier, more-discussed problems. We do not separate those readings. The proxy remains the cutoff-aligned anchor; the modern arm is only a descriptive contrast.

The verdict mix is exact and label-free (Table XIII). All sources are wrong-answer dominated (90–95%), but AI fails to *compile* more often than humans (proxy 6.1%, modern 5.1% vs. 2.6%) and the proxy model crashes more (RE 2.6%); the modern model has the fewest runtime errors (0.5%), consistent with cleaner low-level coding as capability grows. That compile failures survive at all is itself notable: a 6% compile rate under a fixed prompt means the weak model periodically emits C++ that does not build, a failure mode essentially absent from curated human submissions and one that the self-reflection loop only partly clears (RQ4).

### D. RQ4: Self-reflection baseline

We study one concrete, common setup—the proxy model revising its own code with *public example-test* feedback for

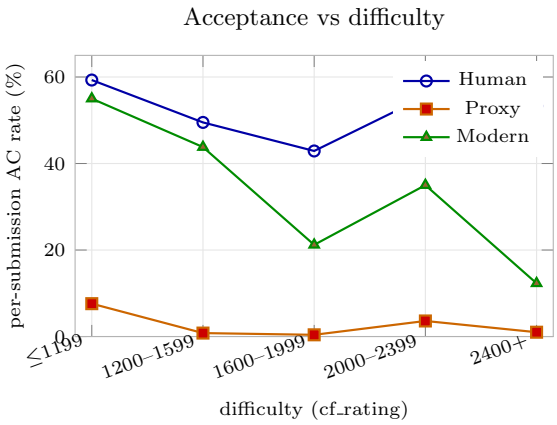


Fig. 3. Acceptance vs. difficulty: the proxy model is at the floor, the modern model declines overall but not monotonically, and humans are flat (survivorship).

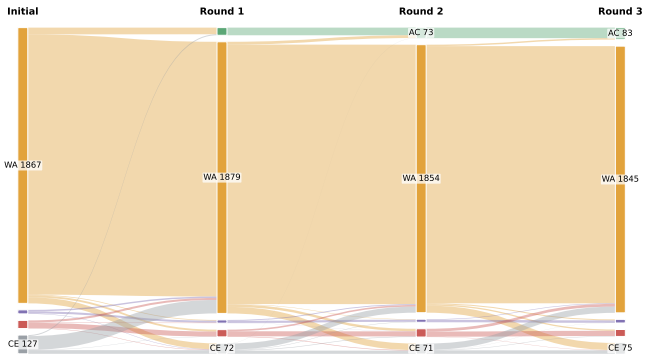


Fig. 4. Verdict transitions of all 2,076 proxy-model bugs across the three self-reflection rounds. The accepted (green) band grows only $0 \to 53 \to 73 \to 83$; wrong answers dominate and persist, and runtime/time-limit failures never resolve.

TABLE XIV
SELF-REFLECTION VS. SAME-BUDGET RESAMPLING FOR PROXY-MODEL BUGS. RESAMPLING ROWS ARE LEAVE-ONE-OUT EXPECTATIONS FROM EACH PROBLEM'S EXISTING 20 CACHED ZERO-SHOT SAMPLES, WITH PROBLEM-BOOTSTRAP 95% CIS; THEY ARE NOT NEW MODEL CALLS.

| Strategy | Extra calls | Fixed | Rate |
|---|---|---|---|
| Self-reflection | 3 repair | 83 | 4.0% |
| Fresh sample | 1 | 37.6 [17.5,61.9] | 1.8% |
| Fresh samples | 2 | 63.3 [29.6,101.5] | 3.0% |
| Fresh samples | 3 | 82.6 [41.6,132.2] | 4.0% |

three rounds. Since those examples are already in the original statement, reflection's new signal is only the model's own wrong output on visible inputs, not hidden-test evidence. It repairs **4.0%** of proxy bugs (83/2,076), which matches a same-budget resampling expectation of **82.6** fixes (4.0%; 95% CI 41.6–132.2; Table XIV). Figure 4 shows most fixes arrive in round 1 ($0 \to 53 \to 73 \to 83$ AC), while wrong answers remain near 1,850. Compile errors reach 5.5% repair and wrong answers 4.1%, but **runtime (0/55) and time-limit (0/27) errors are never repaired**. Using family-level validated judge labels, Fig. 5 shows repairs concentrated in easy syntax/design cases; boundary, math, greedy, and DP failures mostly persist, leaving the diagnostic-label profile almost unchanged.

*a) Repair power collapses with difficulty:* The repair rate falls from **7.1%** on easy problems ($\leq$1599) to **3.4%** on medium (1600–2399) and **1.4%** on hard (2400+). Hard-problem ACs barely move after round 1 ($8 \to 10 \to 10$ of 693), so reflection helps near-miss surface slips, not the design-bound failures that dominate RQ2–RQ3.

## V. DISCUSSION

*a) Benchmark claim boundaries:* HuAIBench separates outcome dynamics, triage-labeler calibration, full-corpus screens, evidence-stability stress tests, and repair baselines. The central result is the gate between them: verdicts support outcome claims directly; the 375-bug gold supports common-evidence labeler evaluation; full-corpus judge predictions support only coarse triage; and root-cause claims require failing-test or reference evidence.

## VI. THREATS TO VALIDITY

**Construct validity.** The human acceptance "rate" is CodeContests' correct:incorrect ratio, so we compare only within-source difficulty trends. Taxonomy labeling is subjective; we use two provenance-blind annotators, adjudication, and report inter-annotator $\kappa = 0.80$ as the reliability ceiling. Common-evidence labels are diagnostic hypotheses, not demonstrated root causes: the 120-item audit changes 55.0% of family labels. RQ1 also shows sensitivity to taxonomy granularity, so we fix a "prefer the most specific leaf" rule and avoid fine causal claims. The judge (`gpt-5.4`) and modern model (`gpt-5.4-nano`) share a model family, and the judge's excess design labels grow across human/proxy/modern (6%/8%/10%); modern-arm taxonomy shifts are therefore screens, not confirmed differences.

**Internal validity.** Excluding generated tests for tractability admits occasional weak-test pass-through, which can mislabel a buggy program as correct; this affects all sources symmetrically. We mitigate special-judge and multiple-output risk by requiring at least one stored correct C++ solution to pass the sandbox comparator, dropping 7 otherwise, and discard unexpected-AC stored incorrects. Verdicts come from a deterministic, cached sandbox and are not subject to taxonomy-labeling error. The modern model's contamination is uncontrolled by design, so its results are a contrast. Contamination can also change the conditional diagnostic-label profile by removing memorized or easy-to-recall AC solutions from the non-AC pool; this is another reason the proxy model anchors cutoff-aligned claims.

**External validity.** We study C++ Codeforces problems, one PaR-style prompt at temperature 1.0, and two models. Because the tasks are public, HuAIBench is not a future-model capability leaderboard; later arms must report contamination status and be interpreted as descriptive cached-arm additions.

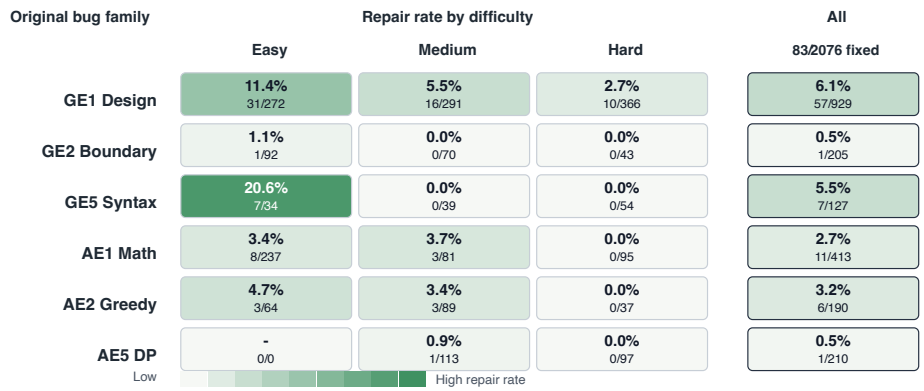


Fig. 5. Taxonomy-aware repair rate of self-reflection, using the original bug's AI-judge primary family label after family-level validation against the 375-item human gold. Rows show families with at least 100 initial proxy-model bugs (two singleton families omitted). Repairs concentrate in easy syntax/design cases and are nearly absent for hard algorithmic families.

The proxy-model limit is *structural*: post-2021 problems require a pre-2021 cutoff, ruling out most stronger code models and motivating contamination-aware benchmarks [7]. Its 3% acceptance is a property of this hidden-test setting, not current LLMs; hopeless attempts can inflate broad math/design diagnoses. WA-only and accepted-problem checks reduce, but do not remove, this confound, so we claim only a bounded coarse profile plus measurement-limited fine shifts.

**Conclusion validity.** Bugs cluster within problems and labels are multi-label, so we bootstrap over the 107 problems rather than treat 14,182 bugs as independent. Some intervals exclude zero, but effect sizes are small and estimated with the same judge whose family-level bias is 6–9 points; corrected-delta CIs in Table IV control our interpretation more than raw significance does. The 125-bug-per-arm gold is underpowered for small shifts ($V \lesssim 0.26$). The stronger-evidence audit adds a coarse check: math/design membership is stable for 70.0% of the audit ([61.3,77.5]) and moves inward more often than outward (25 vs. 11). The dominant coarse region is stable; fine source differences remain measurement-limited.

## VII. Conclusion

HuAIBench measures code failures through explicit evidence tiers: sandbox outcomes, human-gold labeler calibration, full-corpus baseline taxonomy screens, stronger-evidence diagnostic stress tests, and repair baselines. Its first baselines show stable coarse math/design concentration, a syntax/compile separation, and weak public-example self-reflection. The transferable rule is to report results only at the granularity the released evidence can sustain.

## VIII. Artifact Availability

Artifacts: https://anonymous.4open.science/r/aitaxo (metadata, prompts, programs, verdict/test caches, labels, predictions, audit packs, repair trajectories, and scripts).

## References


[1] Y. Li *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[2] M. Wei, Z. Li, X. Chen, M. Zheng, Z. Qu, C. Yu, S. Chen, and X. Ju, "Evaluating and improving LLM-based competitive program generation," *Information and Software Technology*, vol. 191, p. 107977, 2026.

[3] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[5] D. Hendrycks *et al.*, "Measuring coding challenge competence with APPS," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[6] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024.

[8] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[9] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[11] E. Nijkamp *et al.*, "CodeGen: An open large language model for code with multi-turn program synthesis," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] B. Rozière *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[13] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[14] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," in *International Conference on Learning Representations (ICLR)*, 2023.

[15] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of EMNLP*, 2021.

[16] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," *arXiv preprint arXiv:2401.14196*, 2024.

[17] X. Hou *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2024.

[18] T. Y. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024.

[19] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[20] S. Lu *et al.*, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[21] R. Puri *et al.*, "Project CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[22] S. Kulal *et al.*, "SPoC: Search-based pseudocode to code," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[23] O. Sainz *et al.*, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023.

[24] S. Golchin and M. Surdeanu, "Time travel in LLMs: Tracing data contamination in large language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[25] M. Riddell, A. Ni, and A. Cohan, "Quantifying contamination in evaluating code generation capabilities of language models," *arXiv preprint arXiv:2403.04811*, 2024.

[26] I. Magar and R. Schwartz, "Data contamination: From memorization to exploitation," in *Proceedings of ACL*, 2022.

[27] R. Chillarege *et al.*, "Orthogonal defect classification—a concept for in-process measurements," *IEEE Transactions on Software Engineering*, vol. 18, no. 11, pp. 943–956, 1992.

[28] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1665–1705, 2014.

[29] R.-M. Karampatsis and C. Sutton, "How often do single-statement bugs occur? The ManySStuBs4J dataset," in *Proceedings of MSR*, 2020, pp. 573–577.

[30] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.

[31] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proceedings of ISSTA*, 2014, pp. 437–440.

[32] C. Le Goues *et al.*, "The ManyBugs and IntroClass benchmarks for automated repair of C programs," *IEEE Transactions on Software Engineering*, vol. 41, no. 12, pp. 1236–1256, 2015.

[33] M. Tufano *et al.*, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 4, pp. 1–29, 2019.

[34] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768.

[35] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–24, 2018.

[36] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: A survey," *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2019.

[37] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proceedings of ICSE*, 2023, pp. 1482–1494.

[38] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[39] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2023.

[40] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[41] T. X. Olausson *et al.*, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations (ICLR)*, 2024.

[42] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[43] Y. Liu *et al.*, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proceedings of EMNLP*, 2023.

[44] P. Wang *et al.*, "Large language models are not fair evaluators," in *Proceedings of ACL*, 2024.

[45] W. G. Cochran, *Sampling Techniques*, 3rd ed. John Wiley & Sons, 1977.

[46] R. V. Krejcie and D. W. Morgan, "Determining sample size for research activities," *Educational and Psychological Measurement*, vol. 30, no. 3, pp. 607–610, 1970.

[47] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[48] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[49] M. L. McHugh, "Interrater reliability: The kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.

[50] R. Artstein and M. Poesio, "Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008.

[51] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. Sage, 2018.

[52] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

[53] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.